

CONTEXT-AWARE AUTOMATED EXAM GRADING SYSTEM

Comprehensive Technical Architecture & Deep Learning Guide

Optical Character Recognition (OCR) • Qdrant Vector Search RAG • Gemini 1.5 Flash LLM Evaluation

Author / Engineer:	Ujwal Singamsetti
Domain & Specialization:	Generative AI, Retrieval-Augmented Generation (RAG), Natural Language Processing, Computer Vision
Core Technologies:	Python 3.11, FastAPI, Qdrant Vector DB, sentence-transformers, Google Cloud Vision OCR, Google Gemini 1.5 Flash, React 18, PostgreSQL
Project Scope:	End-to-End Handwritten Exam Script Digitization, Vector Curriculum Grounding, Multi-Criteria Scoring, Formative Diagnostics
Document Version:	1.0 (Comprehensive Technical Guide)

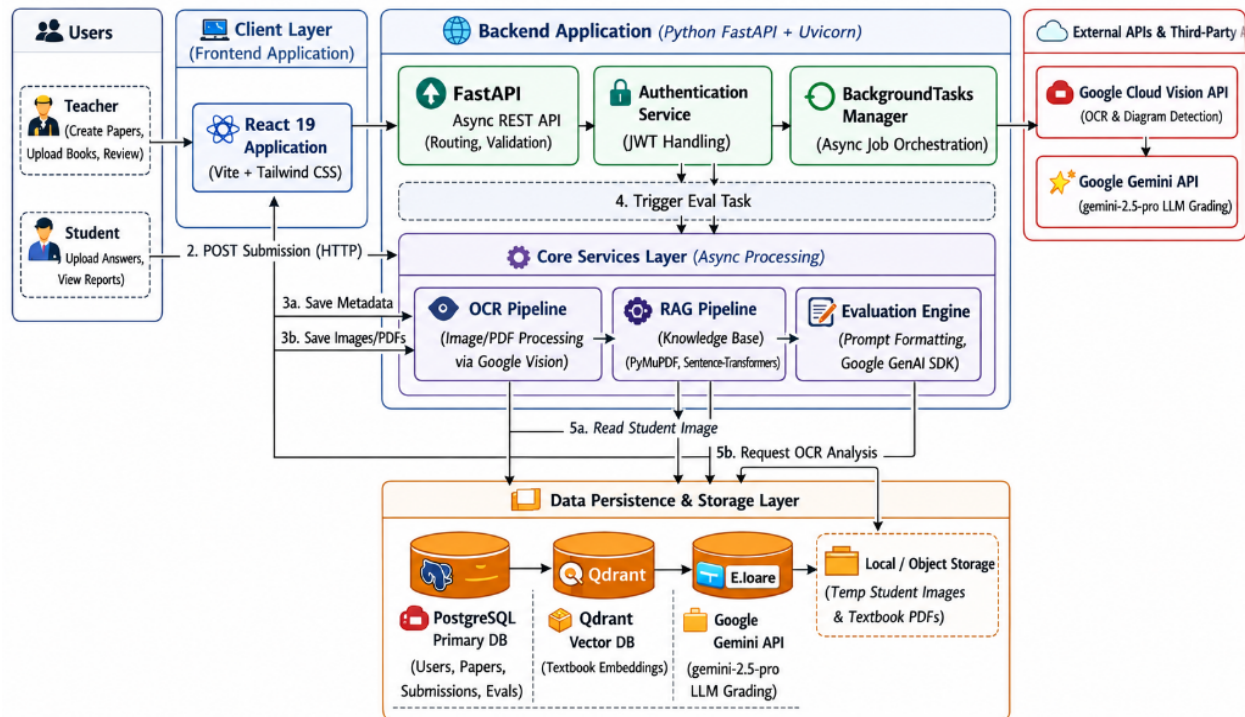


Figure 1.1: System Architecture Diagram of the Context-Aware Automated Grading System

1. System Overview & Problem Domain

1.1 Introduction

Evaluation is an essential cornerstone of academic instruction. However, traditional manual assessment of handwritten descriptive examinations presents major operational hurdles: it is time-consuming, prone to evaluator fatigue, subject to subjective bias, and frequently produces only a single numerical score without offering qualitative diagnostic feedback explaining where marks were lost.

1.2 Problem Statement & AI Engineering Solutions

Existing Automated Short Answer Grading (ASAG) models face three major technical breakdowns:

- 1. **Handwriting & Math Notation Degradation:** Standard text models cannot parse physical handwritten scripts or multi-line mathematical formulas.
- 2. **LLM Hallucinations:** Standalone LLMs lack direct access to specific textbook curricula, leading to incorrect grading or penalizing non-standard yet correct student phrasing.
- 3. **Opaque Numerical Output:** Legacy grading tools fail to provide transparent feedback detailing *what* mistake occurred, *why* it is wrong, and its exact score *impact*.

1.3 Key Architectural Advantages

- **Curriculum Grounding via Qdrant RAG:** Anchors LLM evaluation directly in retrieved textbook chapters, eliminating AI hallucinations.
- **Hybrid Vector & Keyword Re-ranking:** Combines 768-dimensional dense semantic search with domain keyword density matching.
- **Granular Diagnostic Breakdown:** Outputs detailed JSON containing scores, error categories (*what, why, impact*), missing concepts, and model answers.

2. Deep Dive: Technology Stack & Pipeline

Layer / Component	Technology Selected	Technical Function
OCR Engine	Google Cloud Vision REST API	Handwritten text transcription, spatial bounding-box layout parsing.
Embedding Model	all-mpnet-base-v2	Generates 768-dimensional dense semantic vectors.
Vector Database	Qdrant Vector Engine	HNSW vector search with subject and class_level payload filters.
Generative LLM	Google Gemini 1.5 Flash	Grounded prompt evaluation with deterministic JSON output constraints.

Backend API	Python 3.11, FastAPI, Uvicorn	Asynchronous REST service, background task processing, Pydantic validation.
Relational DB	PostgreSQL, SQLAlchemy, Alembic	Persistent storage for user accounts, paper rubrics, submissions, and evaluations.
Frontend UI	React 18 / Vite, Tailwind CSS	Teacher assignment portal, student dashboard, and PDF report card downloads.

2.1 The Hybrid Vector Re-ranking Algorithm

To prevent dense vector search from missing domain-specific technical keywords in student answers, Ujwal engineered a hybrid re-ranking algorithm. Semantic similarity scores from Qdrant are boosted using the keyword match density ratio:

$$\text{Score_boosted} = \min(\text{Score_semantic} + (\text{K_matched} / \text{K_total}) \times 0.2, 1.0)$$

Where `Score_semantic` is Qdrant's HNSW vector score, `K_matched` is the count of matched rubric keywords, and `K_total` is the total target technical keywords.

3. Module Workflows & Visual Diagrams

3.1 Optical Character Recognition & Spatial Layout Parsing

Figure 4.2: Google Cloud Vision OCR Script Parsing & Layout Reconstruction Workflow

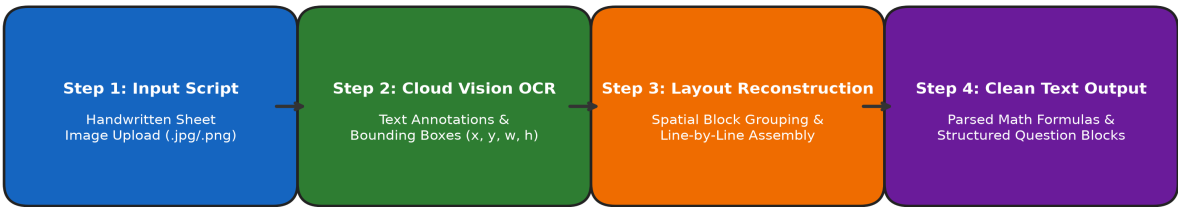


Figure 3.1: Google Cloud Vision OCR Script Parsing & Layout Reconstruction Workflow

3.2 Qdrant Vector Search & Hybrid Re-ranking Pipeline

Figure 4.3: Qdrant Vector Search & Hybrid Keyword Re-ranking Pipeline

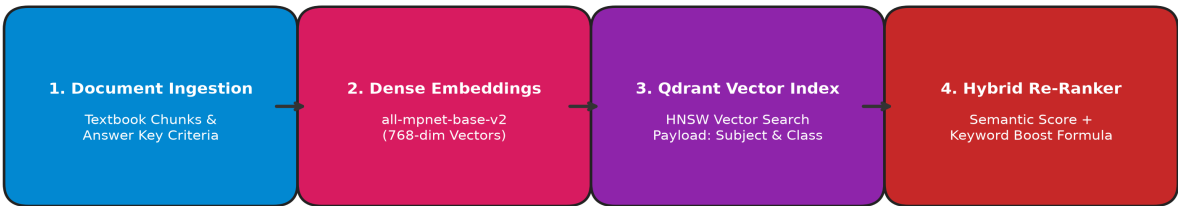


Figure 3.2: Qdrant Vector Retrieval and Keyword Hybrid Re-ranking Workflow

3.3 Gemini 1.5 Flash LLM Evaluation & Diagnostic Parser

Figure 4.4: Gemini 1.5 Flash Evaluation Engine & Diagnostic JSON Generation

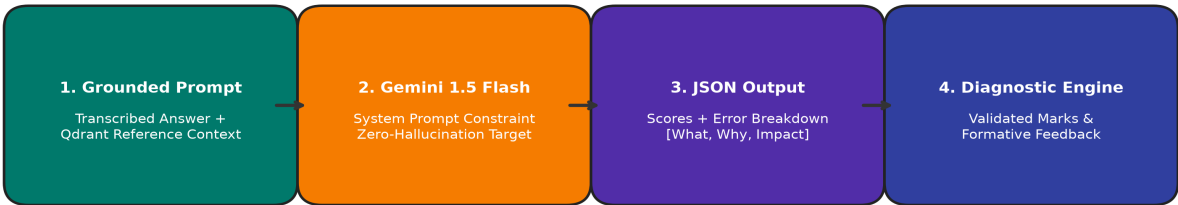


Figure 3.3: Gemini 1.5 Flash Evaluation Engine & Diagnostic JSON Generation Workflow

4. Portfolio Learning Q&A; & Interview Readiness

Q1: How does this project prevent LLM hallucinations during grading?

The system enforces Retrieval-Augmented Generation (RAG). Before sending the student's answer to Gemini 1.5 Flash, Qdrant Vector DB retrieves exact reference passages from indexed textbooks based on subject metadata. The LLM receives strict instructions to grade ONLY against the retrieved reference material.

Q2: Why was Qdrant chosen as the vector database?

Qdrant provides high-throughput HNSW vector indexing, fast sub-15ms similarity search, native support for 768-dimensional embeddings (all-mpnet-base-v2), and payload metadata filtering by subject and class level to isolate search space before retrieval.

Q3: How does the OCR pipeline handle handwritten math formulas?

Google Cloud Vision API REST v1 returns bounding-box spatial coordinates for text blocks. The layout parser reconstructs multi-line text and mathematical expressions by ordering blocks geographically (top-to-bottom, left-to-right).

Q4: What is the diagnostic feedback structure generated by the AI?

Instead of a raw score, every mistake is parsed into 3 dimensions: (1) What error occurred, (2) Why it is scientifically/conceptually incorrect, and (3) Impact on score deduction.

Q5: What are the cost and efficiency metrics of the system?

By combining local Qdrant vector retrieval with Gemini 1.5 Flash token optimization, evaluation cost is under \$0.001 per script page, with sub-second retrieval times.